Jamz.Tech

Sound Recording Management System

Sound Manager Application in C++ – Final Sprint Report

Course: CSE 360 – Software Engineering
Semester: Spring 2026
Instructor: David Claveau
Team: Mary Sannapu, Tanishi Kumar, Anoushka Barua
Repository: https://github.com/MSan789/Jamz.Tech

Abstract

This project is designed to create a Sound Library application that allows stakeholders to search for, record, play, edit, download, and upload their own recorded audio and external audio. Stakeholders can also view their audios in a cluster map, where they can click a cluster to play its corresponding audio. In addition to this detail, the software can be implemented across multiple platforms to maintain continuous access to audios and set prices for each audio.

Introduction

Sound is everywhere, in the music we love, the videos we watch, and the moments we want to remember. However, managing our audio files is often chaotic and confusing. The tools we use today are often built for professionals who are already comfortable with complex software. We aimed to reduce the threshold for audio management.

In this project, we created JamzTech, a sound library application built around one simple idea: finding and managing your audio should feel as easy as listening to it. Whether our stakeholder is a musician keeping track of their latest takes or just someone who records ideas on the go, JamzTech gives you a clean, welcoming space to store, browse, and edit your sounds without any confusion.

From the moment you log in, everything is designed to feel simple and intuitive. Buttons are clearly labeled with images and text, navigation is straightforward, and recordings are always visible. Our stakeholders can search through their library, edit audio, or explore their sounds visually on a cluster map, all without needing any technical background.

We designed Jamz.Tech, because we believe that great tools should always be accessible in the simplest way possible and offer all features so that everyone's choices are included.

Individual Contributions

This page includes details about the individual contributions made by the team members.

Summary of Sprint Velocity: Factoring in the 10 points from in-progress user stories last sprint, we completed 30 points. Our average completion rate for user story points is 5 per sprint, but about 10 per sprint in our last iteration. We have improved this sprint from previous sprints by assigning & completing separate user stories.

Sprint	Key Accomplishments	Story Points
Sprint 1	Account information, login screen	5 pts
Sprint 2	Login, Homescreen Layout, Audio Logic	1 pts
Sprint 3	Database for Audio, Audio List, Recording	5 pts
Sprint 4	RBAC, UI, uploading, price, cluster map, and waveforms.	30 pts

Member	Summary of Contributions	User Stories & Points
Mary Sannapu	*I focused on user role creation & implementation (RBAC), UI designs, pricing & downloading implementation, and uploading audio.* These tasks included creating the two different roles of guests and owners with varying permissions (A2, A4), most noticeably the owner's sole permission to record and upload audio (B1, B6). With Tanishi, I worked on setting prices and downloading audio (C4). I also worked on improving the UI, including waveforms with Anoushka, and additional features.	A2: 2 A4: 3 B6: 2 C4: 3
Anoushka Barua	*I focused on integrating waveform viewing, optimizing UI features, and fixing minor bugs.* These tasks included creating waveforms during live recording and on each track (B1, B4, and C1). Waveforms were implemented on our record page, each track on the homescreen, and adjusted accordingly to meet requirements in collaboration with Mary. Certain buttons and functions were fixed, such as returning to the home page after the edit window pop-up and the recording page.	B1: 5 B4: 3 C1: 1
Tanishi Kumar	*I focused on creating a cluster map and implementing audio editing, price implementation, and the edit feature for the app.* These tasks included creating a functional cluster map feature that displays all audio recordings in a clustered format based on their x- and y-axes (C2, B5). Additionally, I worked on creating the editing features, such as volume and pitch adjustments (B3, C3), in collaboration with Mary. I also made small changes to the app's features, adding a "favorites" section under the categories panel that displays all audio files users have favorited. In collaboration with Mary, the price feature was set up and enhanced to allow owners to set prices and for both users and owners to download (C4).	B5: 5 B3: 5 C2: 2 C3: 3 C4: 3

System Description

High-level description of what the application does, user roles (Owner / Guest), key features, technologies used, and architecture using screenshots.

Backlog

This section includes the project backlog

Future improvements: RBAC between Owner and Guest, Editing Sound ability, Downloading sound ability, Cluster Map.

USER STORY NUMBERING SYSTEM

Story Points in parentheses

A. New Users

1. As the first user of the system to log in, I can establish my username, password, and account information using a special screen for the first user. (3)
2. As the first user, I am assigned the role of an owner. (2)
3. Once I have finished providing the required account information, I'm required to log in again. (2)
4. As a user of the system, I can be assigned the role of owner or guest. (3)

B. Owner

1. As an owner, I can record new sounds using the built-in microphone on a laptop and view the sound's waveform. (5)
2. As an owner, I can play a sound. (2)
3. As an owner, I can filter the sound to change its pitch, its length, and possibly other effects. (5)
4. As an owner, I can save new sounds and view a list of all previously saved sounds. (3)
5. As an owner, I can view all saved sounds as a 2-D map of clusters/clouds of dots (each dot represents a sound, and each cluster represents a grouping based on similarity). (5)
6. As an owner, I can create a guest account. (2)

C. Guest

1. As a guest, I can view all sounds as a list. (1)
2. As a guest, I can view the 2-D cluster map of sounds. (2)
3. As a guest, I can play a sound and apply filtering to it. (3)
4. As a guest, I can download a sound after agreeing to the terms of use and a price. (3)

Plan

This section includes plans & layouts

Kanban Board

Uses the software ClickUp, an interactive collaborative tool for teams

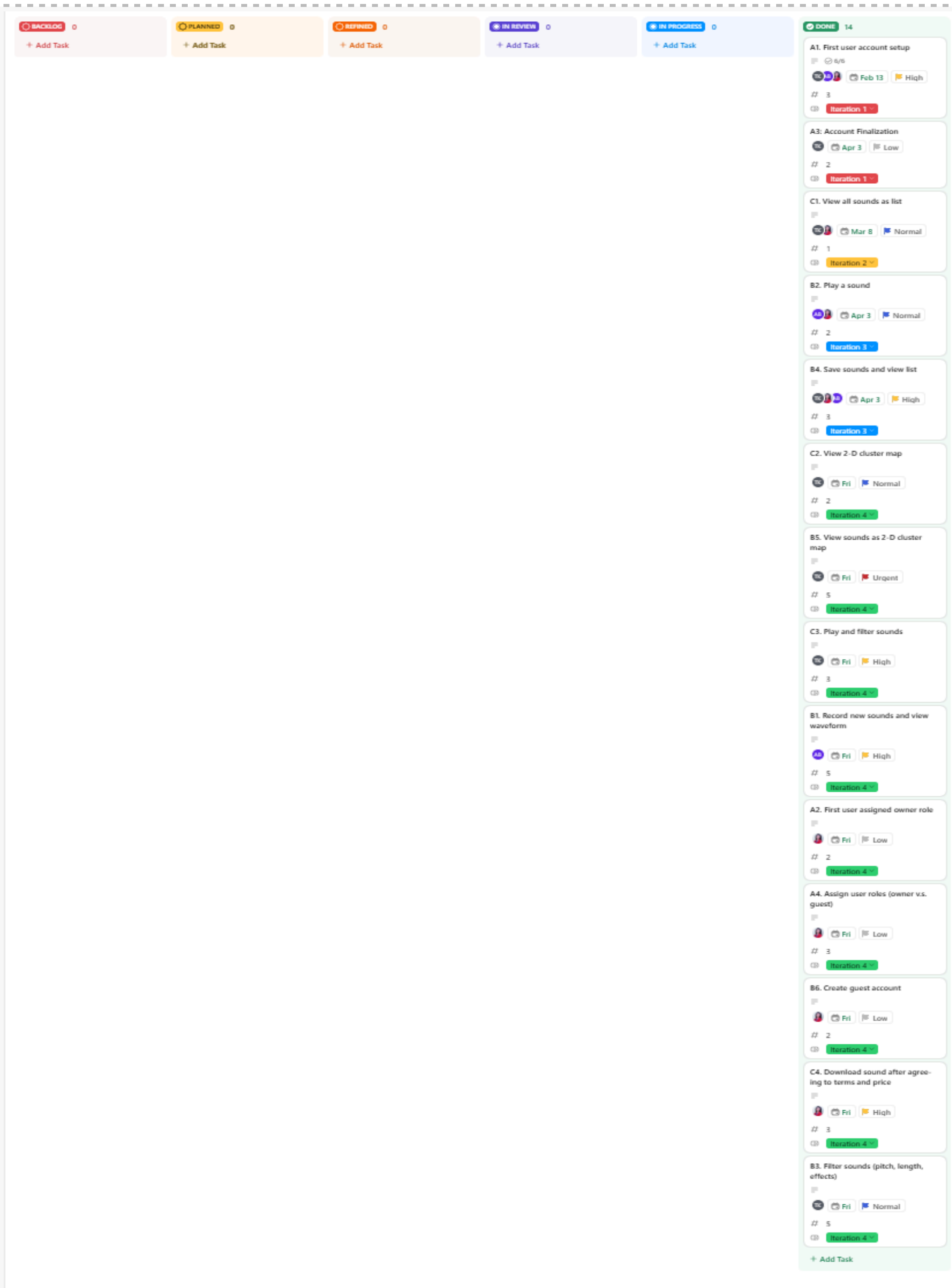


Figure 1: Current Backlog of Project using Kanban

Gantt Chart

Uses the software ClickUp, an interactive collaborative tool for teams

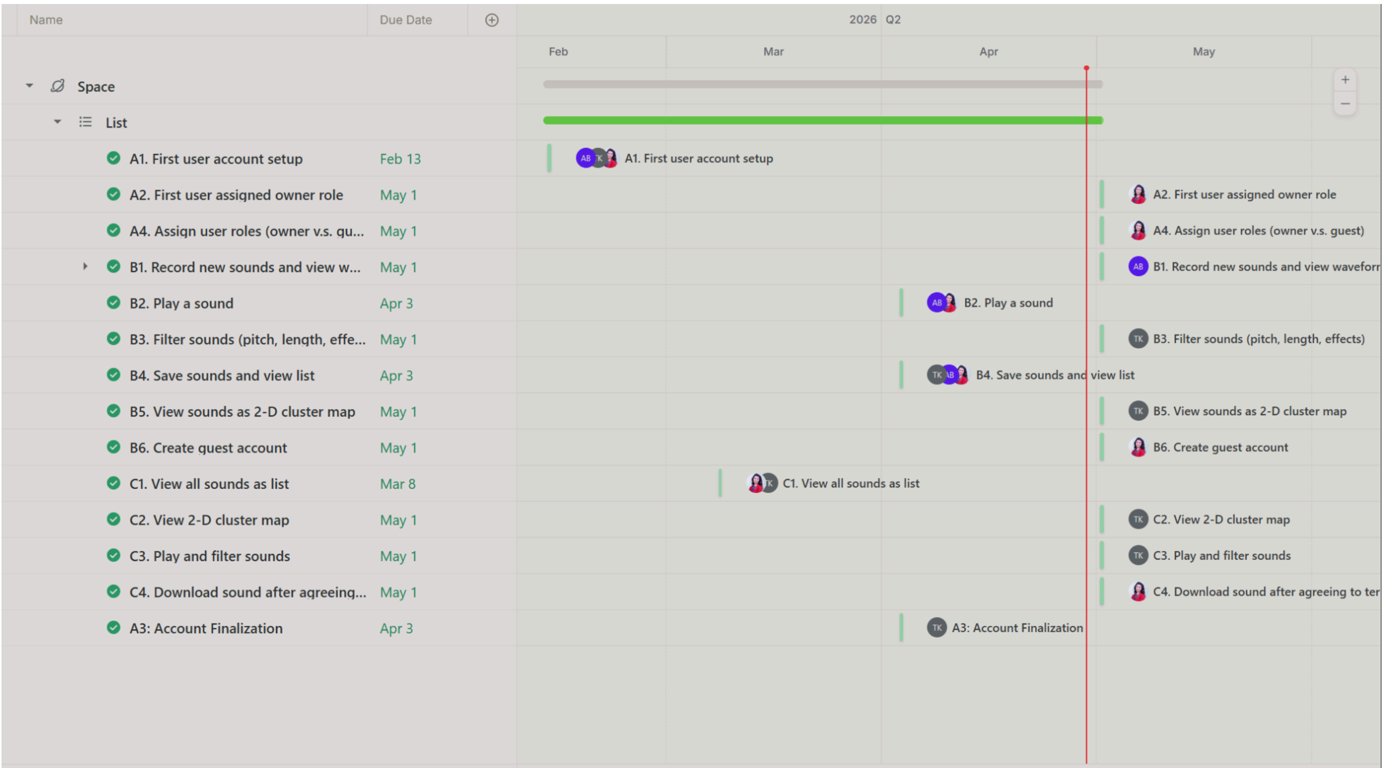


Figure 2: Current Gantt Chart of Project

UML Diagram

Lays out the main structure of the software application

Figure 3: UML Diagram

Technologies:

Technology	Purpose	Notes
JUCE	*Interface*	*JUCE is an audio interface that includes audio modules for use*
CppUnit	*Test Cases*	*A unit testing framework for the C++ programming language*
GitHub	*Version Control*	*A central repository to store, manage, share, and collaborate on code projects*
C++	*Main programming language*	*A programming language that can handle everything from low-level hardware communication to high-level application development*
ClickUp	*Backlog and Planning*	*A cloud-based platform designed to centralize tasks, documentation, and team collaboration into one workspace*
Mermaid.js	*UML Design*	*A tool to create and modify diagrams dynamically, facilitating easy diagram creation.*

Results & Outputs Walkthrough

This section includes screenshots of the application.

When a user first opens our software, they are prompted to the sign-in page, where they can log in. The user would then decide to either log in with a Guest or an Admin account to access our software. If the user were to log in with a Guest account, they would be shown the Guest UI, and vice versa for the admin account.

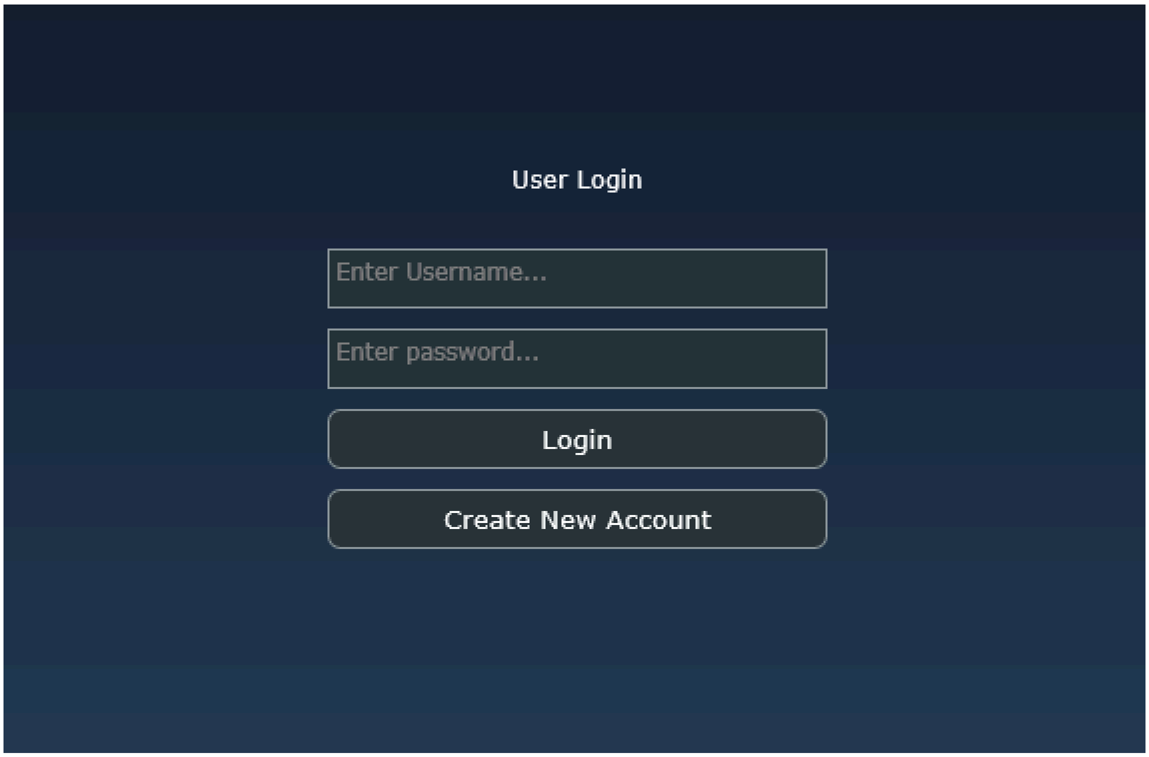
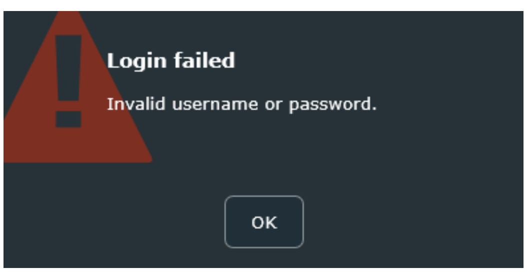


Figure 4: Login UI for users

Users can also reset their password if they forget their username and password. They would first click the "Forgot credentials?" button, then follow the steps provided to reset their account credentials based on their situation.



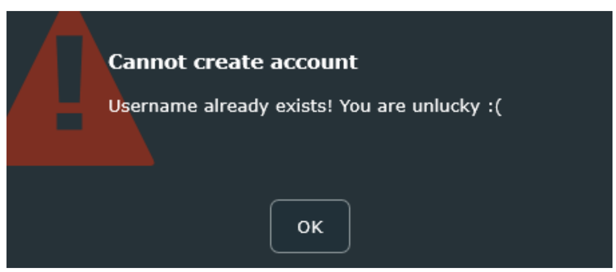


Figure 5: "Invalid Credentials" pop-up and Figure 6: "Username Contradiction" window.

Users can also create a new account by clicking the "Create Account" button in Figure 5 and entering the required information, as shown in Figure 5.

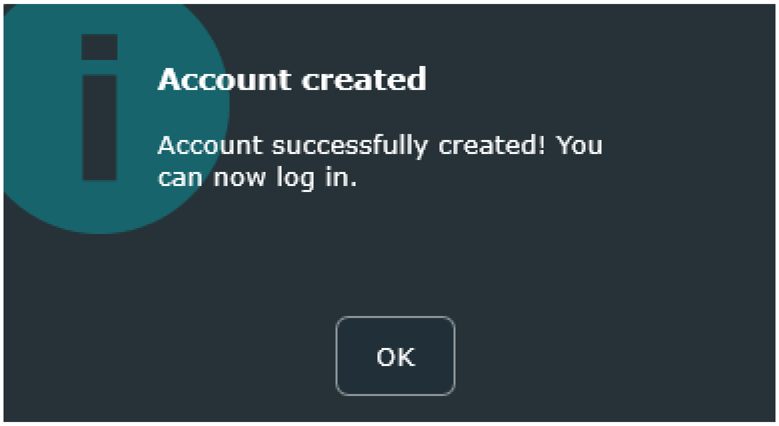


Figure 7: "Account Created" Popup

Users who log in/create new accounts on the main screen are automatically assigned the Owner role, which gives them the ability to record and upload new sounds, create guest accounts, and use all other app functionality.

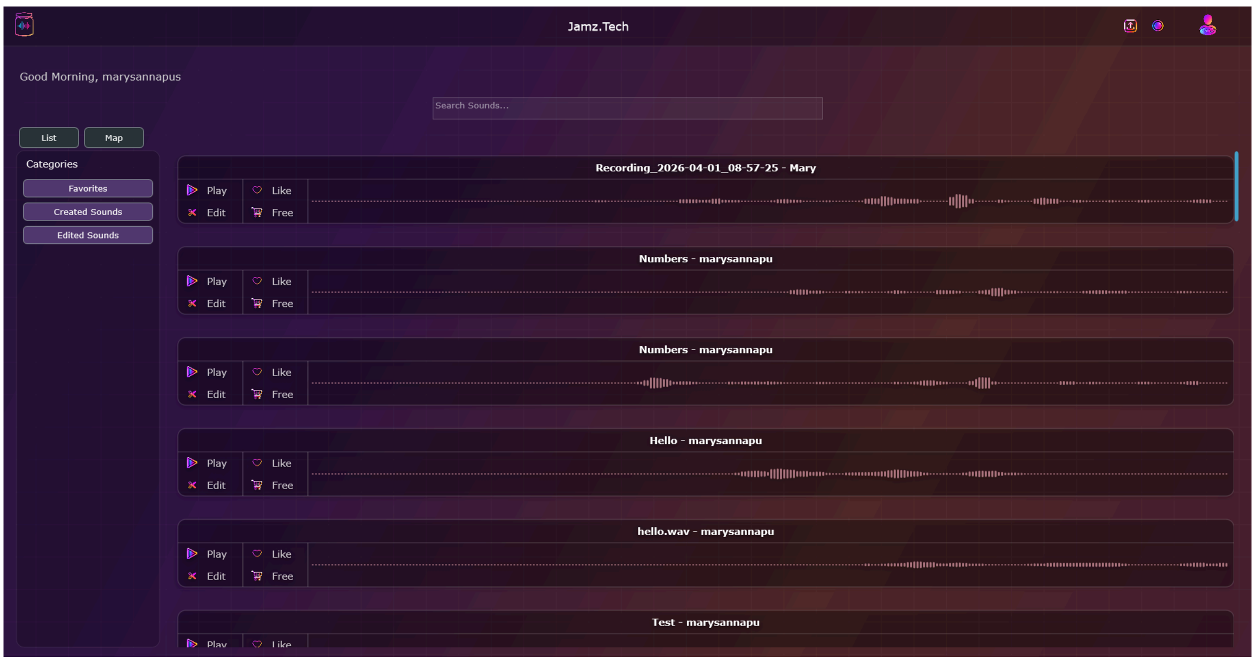
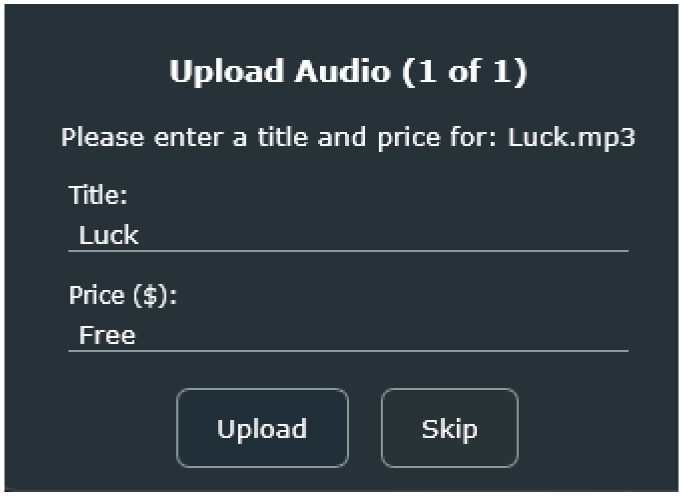


Figure 8: Owner UI

The owner can upload audio from their local device by clicking the upload button in the header bar. They can set the title and price for each uploaded audio file.



,

Figure 9: Upload Audio

Owners can also record sounds by clicking the recording button in the header bar. They can pause, continue, play back, delete, and save their recording. When owners save their recording, they can set the title and price.

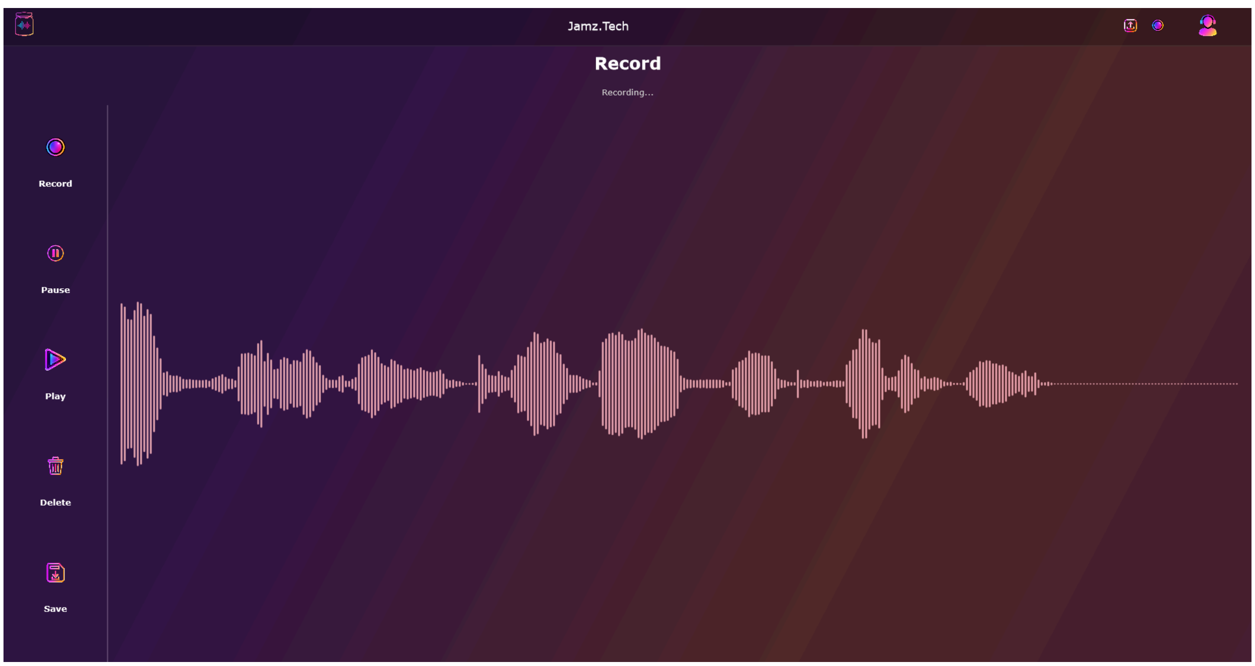


Figure 10: Upload Audio

Figure 11: Saving Audio Recording

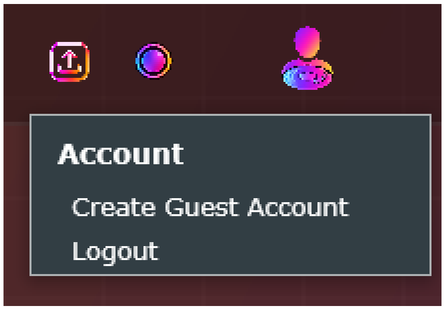


Figure 12: Account Dropdown

Owners can create a guest account in the UI. Guests have all application abilities, except the ability to record and upload sounds.

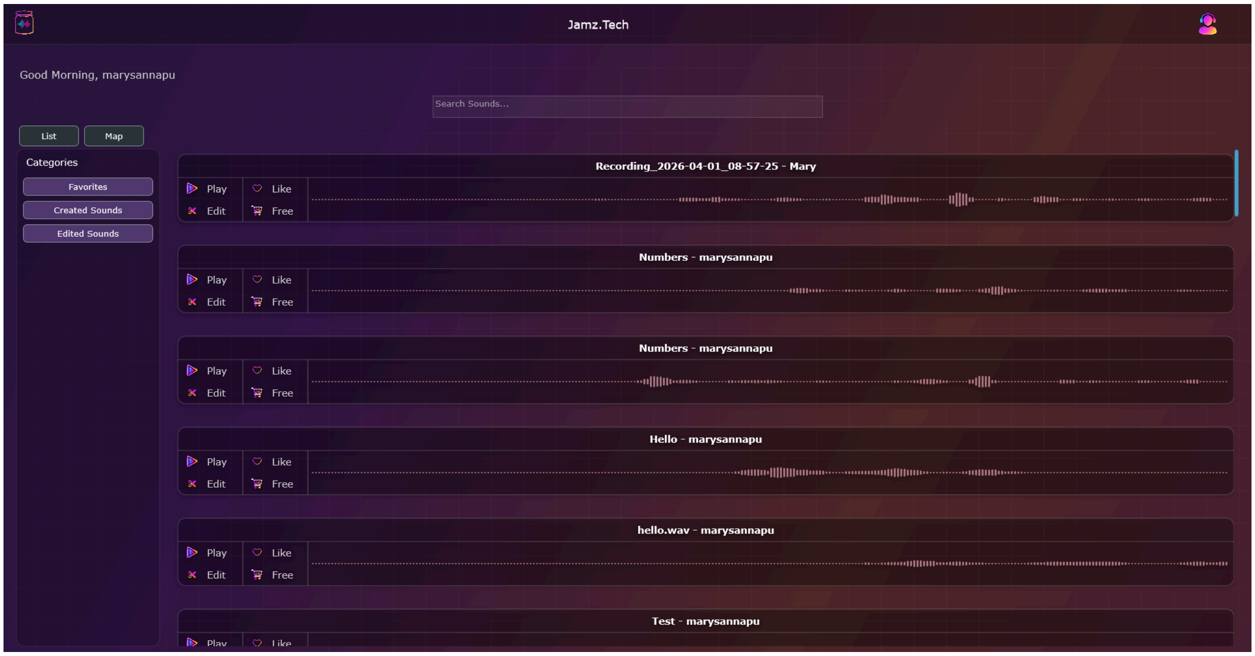


Figure 13: Guest View

All the following features are accessible for both guests and owners. At any time, clicking on the logo on the top right will take the user back to the sound list view homescreen. All sounds display a waveform, title, name of the account that created/uploaded the audio, and several features, including playback, favorite, edit, and buy. Users can search for audio.

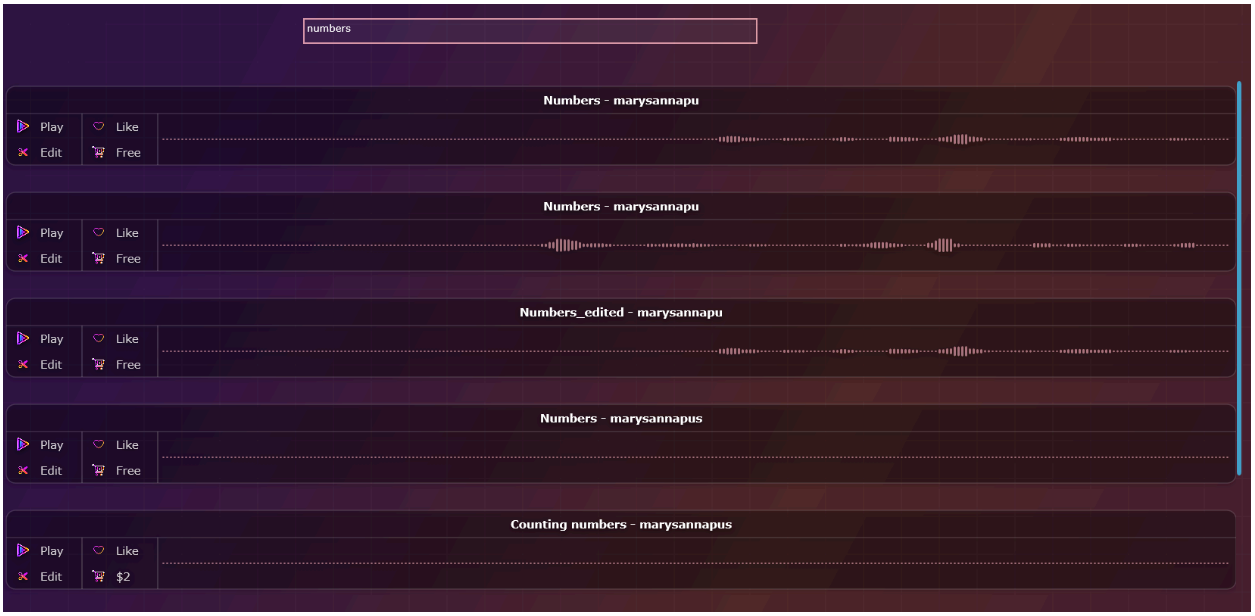


Figure 14: Search Functionality

By clicking the “like” button on list view, users can access their “favorite” list of audios by clicking the “favorite” filter in the “categories” side panel.

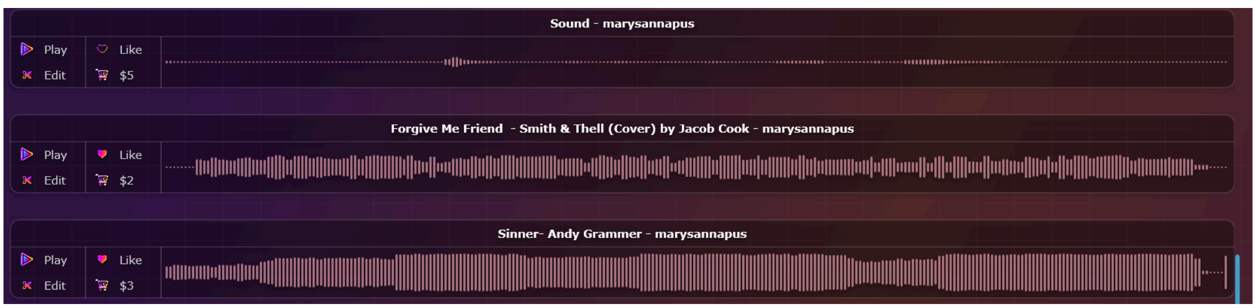


Figure 15: Favorited view of audios

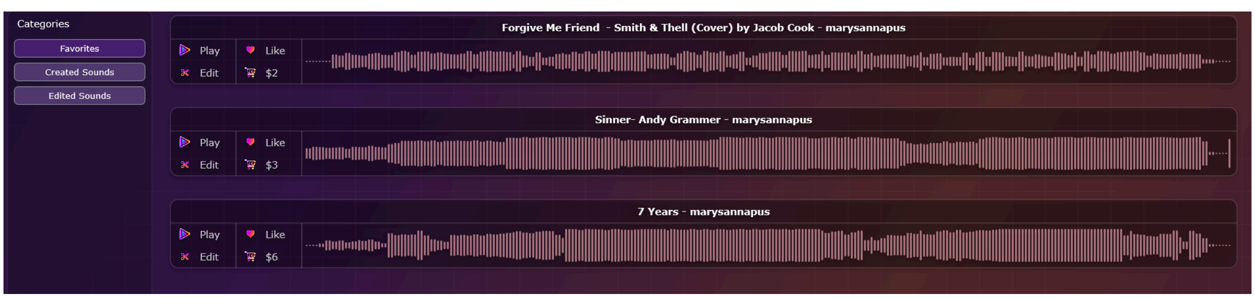


Figure 16: Favorite filter applied

Users can click on “created sounds” or “edited sounds” to see the appropriate filter applied to the list view of sounds they personally created/uploaded or edited.

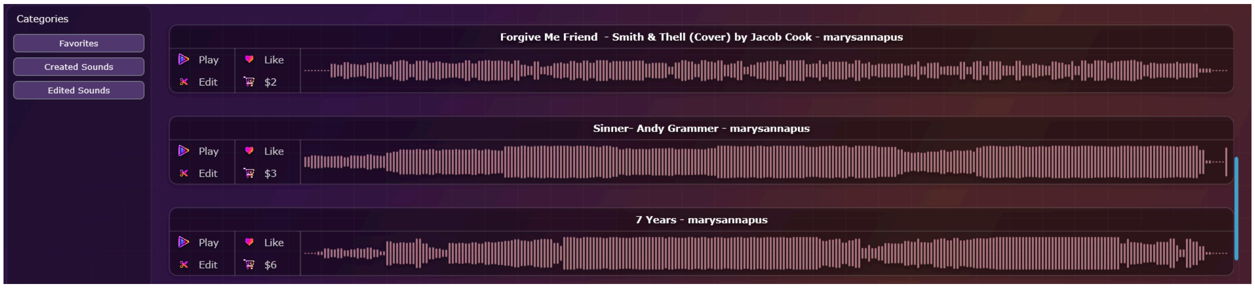


Figure 17: Created Sounds filter applied

Figure 18: Edited Sounds filter applied

Users can also view a cluster map of audio recordings instead of a list view by clicking the "map" button in Figures 9 & 13. They can return to the list view by clicking the "list" button. The cluster map is designed so that the x-axis represents the audio files by their naming conventions, and the y-axis arranges them from oldest (at the bottom) to newest (near the top). Clicking a cluster or dot plays the corresponding audio.

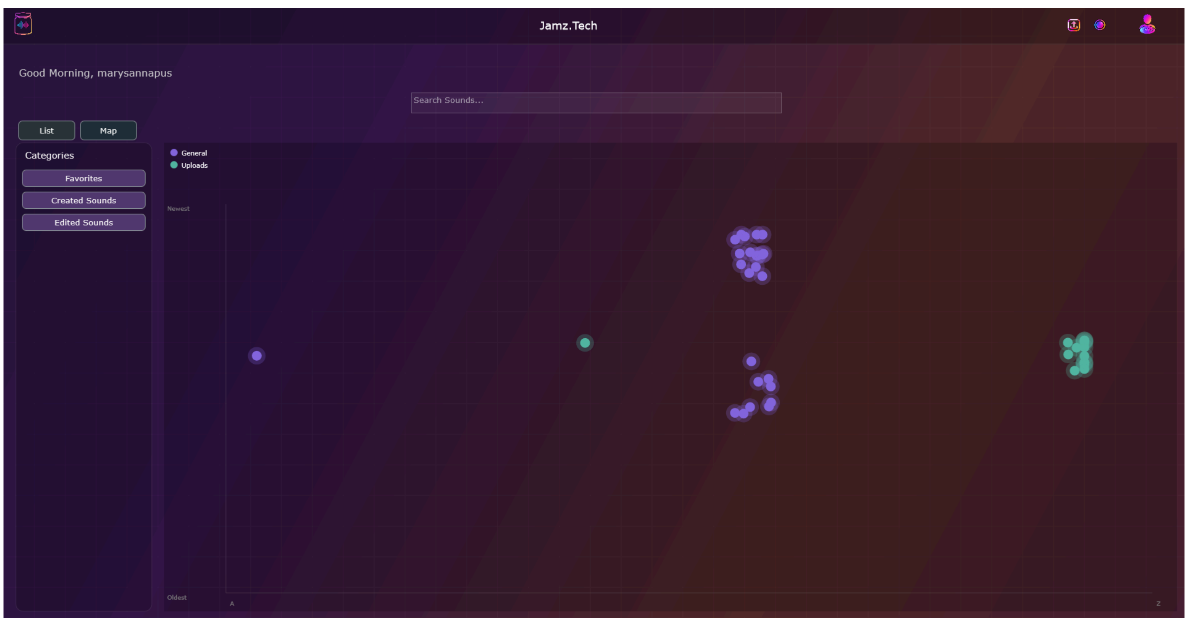


Figure 19: Cluster Map View

Figure 20: Highlighted view of a singular audio data point

Clicking on the "edit" button on the audio cards, as displayed on Figures 9 & 13, will allow the user to "edit" audios. They can modify pitch, volume, and reverb. They can also pause, play, save, and delete their audio. Saving their audios will create a "remix" version of the audio, not delete the original.

Figure 21: Edited Sounds View

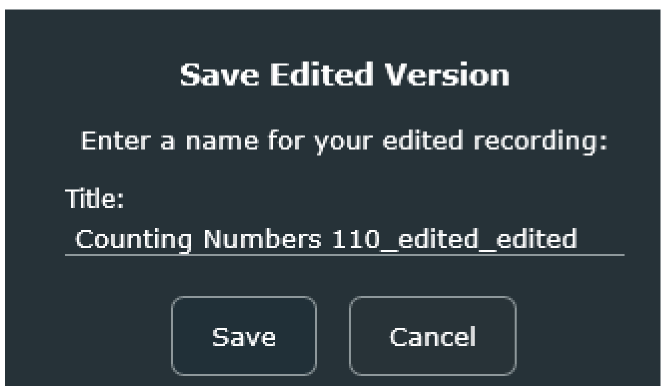


Figure 22: Saving new edited sound

After agreeing to the terms and price, users can click the buy button on audio cards as present in Figures 9 & 13, and download the audio to their local computer.

Figure 23: Downloading purchased sound

Tests

This section includes tests

To evaluate the functionality of Jamz.Tech application methods, we put it through a thorough round of testing, covering the major sections of the app that play a key role in its features and functionality. We have created tests to evaluate the login logic, homescreen functionality, recording functionality, local database creation, and cluster map functionality. These structured test cases are run automatically and display test outputs in the tester's terminal before loading the application, revealing any issues and all passed tests.

---

5 SECTIONS of code made to check the functionality of the functions that made our application:

These cover user stories like: A1, A3, B1, B2, B4, C1, and B5.

<u>LOGIN LOGIC TESTING</u>

(User Stories: A1, A2, A3)

Description: This section of the code tests if a new user can create a new account successfully. In addition, users with a valid username and password will only be able to log in successfully. Any empty fields will cause errors for users. This section of testing will give a "test failed" output if it fails to meet code expectations on the code end; otherwise, the tests will simply get completed:

Expected output/Received output on terminal (based on the test) and when it got tested:

Starting tests in: Login Logic Tests / Create a new account...
Completed tests in Login Logic Tests / Create a new account

Starting tests in: Login Logic Tests / Duplicate username is rejected...
Completed tests in Login Logic Tests / Duplicate username is rejected
----
Starting tests in: Login Logic Tests / Login succeeds with correct username and password...
Completed tests in Login Logic Tests / Login succeeds with correct username and password
----
Starting tests in: Login Logic Tests / Login fails with wrong password...
Completed tests in Login Logic Tests / Login fails with wrong password
----
Starting tests in: Login Logic Tests / Login fails with unknown username...
Completed tests in Login Logic Tests / Login fails with unknown username
----
Starting tests in: Login Logic Tests / Owner role is stored and loaded correctly...
Completed tests in Login Logic Tests / Owner role is stored and loaded correctly

----
Starting tests in: Login Logic Tests / Empty username or password is rejected before account creation...
Completed tests in Login Logic Tests / Empty username or password is rejected before account creation

HOMESCREEN LOGIC TESTING

(User Stories: B4)

Description: This section of the code will test if verified users, upon login, are able to access the homescreen or not. In addition, all recorded audio files must be loaded and be visible successfully. The "on click" button functions should work appropriately and should redirect the users to the specified pages. Again, if failure is detected, the tests won't complete successfully and will stop; otherwise, the tests will be completed successfully. This section tests all of this:

Expected output (upon success and completion) on IDE console:

Starting tests in: Homescreen Tests / Homescreen constructs successfully...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Completed tests in Homescreen Tests / Homescreen constructs successfully
----
Starting tests in: Homescreen Tests / setUsername stores the username...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Completed tests in Homescreen Tests / setUsername stores the username
----
Starting tests in: Homescreen Tests / loadRecordings creates cards only for the logged-in user...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Data inserted successfully.
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Completed tests in Homescreen Tests / loadRecordings creates cards only for the logged-in user
----
Starting tests in: Homescreen Tests / AudioCards onPlayClicked callback is invoked...
Completed tests in Homescreen Tests / AudioCards onPlayClicked callback is invoked
----
Starting tests in: Homescreen Tests / AudioCards onEditClicked callback is invoked...
Completed tests in Homescreen Tests / AudioCards onEditClicked callback is invoked
----
Starting tests in: Homescreen Tests / AudioCards getRecording returns the assigned entry...
Completed tests in Homescreen Tests / AudioCards getRecording returns the assigned entry

RECORD COMPONENT TESTING

(User Stories: B1, B2)

Description: This section of testing will test out the code/logic/functionality of the recording aspect of our application. Users should be redirected successfully when they click the record button. In addition, users should be able to record their audio. They should be able to STOP their recordings. They must also be able to save their audiofiless in a file format on their local computer (which will later be handled with our SQL logic testing as well). Upon refreshing the homescreen, all audios on the local computer of the user has recorded must be visible.

Expected output on the console if tests were successful and things are functional:

Starting tests in: RecordComponent Tests / RecordComponent constructs without error...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Completed tests in RecordComponent Tests / RecordComponent constructs without error
----
Starting tests in: RecordComponent Tests / setAccountName does not crash with any string...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Completed tests in RecordComponent Tests / setAccountName does not crash with any string
----
Starting tests in: RecordComponent Tests / Filename sanitisation removes illegal characters...
Completed tests in RecordComponent Tests / Filename sanitisation removes illegal characters
----
Starting tests in: RecordComponent Tests / Recordings output folder can be created...
Completed tests in RecordComponent Tests / Recordings output folder can be created
----
Starting tests in: RecordComponent Tests / onBack callback fires when assigned...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Completed tests in RecordComponent Tests / onBack callback fires when assigned
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.

LOCAL DATABASE TESTING

(User Stories: B1, B2 – makes sure audios are loaded in the database)

Description: This section of the code tests the code functionality of being able to successfully create tables using the SQL structure. Part of our application needs to be able to access all audios in table format from a single location for the owner. We want to make sure recordings are accessible and retrievable. This code testing verifies this aspect of our application:

Expected output if tests run successfully on the IDE console:

Starting tests in: LocalDatabase Tests / Database initialises without error...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Completed tests in LocalDatabase Tests / Database initialises without error
----
Starting tests in: LocalDatabase Tests / Insert and retrieve a recording entry...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Data inserted successfully.
Completed tests in LocalDatabase Tests / Insert and retrieve a recording entry
----
Starting tests in: LocalDatabase Tests / Multiple inserts are all retrievable...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Data inserted successfully.
Data inserted successfully.
Completed tests in LocalDatabase Tests / Multiple inserts are all retrievable
----
Starting tests in: LocalDatabase Tests / Retrieved entries have non-empty required fields...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Data inserted successfully.
Completed tests in LocalDatabase Tests / Retrieved entries have non-empty required fields

CLUSTER MAP TESTS:

(User Stories: B5, C2)

Description: This section of the code tests the functionality of the cluster map feature. The first test is created to ensure that the cluster map feature is always created and included when the application loads. For the second test, the database and recordings page should be operating perfectly, loading everything into the cluster map so the cluster map can display and play the corresponding audio files. Additionally, we want to make sure the cluster map doesn't crash or cause the application to error. Lastly, we make sure the variable/method for setSearchQuery in this section is working properly and not causing any internal errors before the application loads.

Expected outputs of the tests if everything is functional:

Starting tests in: ClusterMap Tests / ClusterMapComponent constructs without error...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db

Tables created successfully.
Completed tests in ClusterMap Tests / ClusterMapComponent constructs without error
----
Starting tests in: ClusterMap Tests / loadRecordings does not crash on empty or populated database...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Completed tests in ClusterMap Tests / loadRecordings do not crash on empty or populated database
----
Starting tests in: ClusterMap Tests / setSearchQuery handles arbitrary input without crashing...
Opened database at: /Users/tanishi/Library/JamzTech/recordings.db
Tables created successfully.
Completed tests in ClusterMap Tests / setSearchQuery handles arbitrary input without crashing

Results & Discussion

The results of our software and how we achieved our goals

Our results were as expected, though we have encountered bugs and runtime errors that we had to fix quickly to ensure smooth execution. Tasks were assigned and delivered on time, and additional features were implemented to enhance the user experience beyond expectations. Sprints delivered new features and improvements every Wednesday, and feedback was incorporated to improve them. Major UI changes were made to create a dynamic interface and increase accessibility.

Conclusion

In conclusion, our sound library application fulfills the requirements for a simple, user-friendly platform for managing and interacting with audio content. The Jamz.Tech application is easy for users to find, play, and edit sounds without any trouble or confusion. The application consists of two different user roles- guest and owner, which provide structure to the system with limited features for guest users while allowing flexibility. Users can establish a new username, password, and account information on a pop-up screen. New users are automatically assigned as owners, who have access to recording, editing/modifying, uploading, saving, setting a price on each track, and favoriting tracks. The owner can also view and play sounds on the 2-D cluster map. Guest accounts are created by owners, allowing guest users to view sounds as a list, view a 2-D cluster map, play and apply filters to available tracks, and download sounds. Although these features meet the requirements, the application can be improved by integrating a larger database for smoother functionality, enabling access to tracks from any machine via a cloud database, and adding more features to optimize the application and make it more user-friendly. Overall, this application demonstrates an effective, accessible sound management system through a focus on usability and thoughtful design.

Acknowledgments

Dr. Claveau has provided routine feedback over the semester, which we have incorporated to improve our app; we have also sought our peers’ feedback, which has provided helpful tips for navigating bugs and features.